

SE Lab 3: Component Modelling & Architectural Pattern Selection

Written Justification | Problem Statement #17: Vaccination Cohort & Dose Scheduling System
Shrihari Nagesh | PES1UG24CS445 | Dept. of CSE, PES University

Architecture Selection

We chose **Microservices Architecture** for the Vaccination Cohort & Dose Scheduling System.

Style	How it fits this system
Layered	Simple to build, but it is one deployable unit. A booking rush would force us to scale the whole application, and one fault can take down every function.
Client-Server	Central control and easy data consistency, but the single server becomes a bottleneck and a single point of failure during a rollout.
Microservices (chosen)	More moving parts to run, but each function scales and fails on its own. That suits a system whose load is very uneven.

Reason 1: The load is uneven, so parts must scale separately

When a new cohort opens, thousands of citizens book at the same moment (NFR-003: 5,000 concurrent booking requests, 95% answered within 3 s). Dose recording and certificate checks stay at their normal volume. With microservices we add instances of only the Slot Booking Service for the rollout window, instead of copying the whole system.

Reason 2: A failure in one area must not stop vaccination at the centre

Booking, dose recording and certificate checks are separate services, each with its own database. If the Slot Booking Service is overloaded or down, Vaccination Officers can still record doses (FR-004) and certificates can still be verified. In a layered or client-server design the same overload would block all of these together.

Security Advantage

The private key that signs QR certificates is held only inside the Certificate Service. The public-facing services (registration and booking) and their databases have no access to it, so an attacker who breaks into the booking service still cannot forge a vaccination certificate. All outside traffic also enters through the API Gateway, which checks the login token and the role (citizen or officer) before any service is reached, as NFR-002 requires.

Performance Benefit

Certificate verification runs as its own small, stateless service, so it never waits behind booking traffic. QR verification can stay within the 150 ms target (NFR-001) even during a booking peak, while the booking service scales out on its own to hold its 3 s target.

Trade-off Accepted

Microservices add network calls between services and more deployment work. We limit this by keeping slot reservation inside one service and one database, so double booking is prevented by a single local transaction. Only four interactions cross between services (cohort lookup, dose-interval check, certificate issue, notification), and two of them are asynchronous.

Diagram: Architecture_Diagram.pdf shows 8 components, 4 per-service data stores and 10 interfaces in ball-and-socket notation.